PI Conception d'un Robot Intelligent

DIOP Amsatou EL OMARI BOUYA Lina GUÉNOT Tom
LEVASSEUR Nathan VESPIER Michel

6 janvier 2025

Objectif

Concevoir et construire un robot suiveur de ligne capable de sortir d'un labyrinthe le plus vite possible

Ce projet a pour objectif de concevoir un robot autonome capable de sortir d'un labyrinthe. Il s'inscrit dans une démarche visant à approfondir nos connaissances en algorithmique, en programmation en langage C, et en électronique, tout en nous initiant aux pratiques collaboratives, essentielles dans le cadre de notre formation d'ingénieur.

Le robot sera doté de capteurs variés lui permettant de collecter des informations sur son environnement et d'exécuter les instructions programmées à l'avance. À partir d'un plan de labyrinthe donné en entrée, les algorithmes développés auront pour mission de générer un chemin vers la sortie, ainsi que les ordres nécessaires à l'exécution de ce trajet par le robot.

Ce compte rendu présente les différentes étapes du projet : une première partie sera consacrée à la conception algorithmique et électronique, tandis qu'une seconde partie abordera les problématiques rencontrées au cours de ces phases.

Table des matières

1 Informatique **2**
1.1 Analyse . . . 2
1.1.1 Explications TAD . . . 2
1.1.2 Analyse Descendante . . . 2
1.2 Conception préliminaire . . . 3
1.2.1 Conception Préliminaire Labyrinthe . . . 3
1.2.2 Conception Préliminaire Case . . . 3
1.2.3 Conception Préliminaire ordresRobot . . . 4
1.3 Conception détaillée . . . 4
1.3.1 SDD Labyrinthe . . . 4
1.3.2 SDD Case . . . 6
1.3.3 Conception Détaillée ordres . . . 9
1.4 Tests . . . 15
1.5 Retour sur la partie algorithmique . . . 15

2 Électronique **16**
2.1 Description des composants . . . 16
2.2 Schéma électrique . . . 18
2.3 Positionnement des capteurs . . . 20
2.4 Algorithme de déplacement . . . 21
2.5 Problèmes recontrés . . . 22

3 Voies d'amélioration **24**

1 Informatique

1.1 Analyse

1.1.1 Explications TAD

TAD Case Le TAD Case a été créé afin de pouvoir uniformiser l'accès aux données des cases voisines.

Celui-ci permet ainsi de connaitre les différentes cases qui lui sont accessibles, et avec leur direction.

Afin de ne pas surcharger les fonctionnalités, nous avons atomisé les actions possibles. De ce fait, nous avons donc `obtenirCaseHaut`, `obtenirCaseBas`, `obtenirCaseGauche` et `obtenirCaseDroite`. C'est ainsi une simple interface d'accès aux données stockées. Aucune réelle logique métier supplémentaire est ajouté ici.

TAD Labyrinthe Le labyrinthe est un regroupement de cases. Il est en quelque sorte une surcouche de logique métier au TAD Case. Il contient cependant une information supplémentaire, celle des cases d'entrée et de sortie.

La valeur ajoutée au TAD Case qu'il englobe est la possibilité de récupérer tout les voisins d'une case. L'algorithme de recherche de plus court chemin a été séparé du TAD Labyrinthe, puisqu'il n'a pas été vu comme indispensable à une existence d'un labyrinthe.

1.1.2 Analyse Descendante

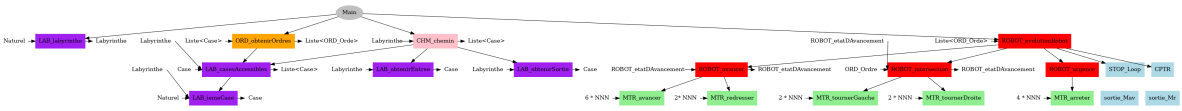

Table 1 – Schéma électrique

Le programme commencera par exécuter `LAB_labyrinthe` pour créer un labyrinthe. Il se basera sur la lecture en entrée standard pour en dessiner l'architecture. Le labyrinthe sera ensuite utilisé pour trouver un chemin vers la sortie. Pour cela, nous utilisons `CHM_chemin`. Un algorithme de plus court chemin avec Dijkstra a été envisagé, mais non implémenté. Cette fonction, à travers l'utilisation de `LAB_obtenirEntree` et `LAB_obtenirSortie`, et des appels en boucle de `LAB_casesAccessibles`, nous donnera la liste des cases à parcourir de l'entrée

à la sortie. Avec ce chemin trouvé, nous le donnons à `ORD_obtenirOrdres`, qui génèrera la liste des ordres que le programme de pilotage du robot comprendra.

`ROBOT_evolutionRobot` récupérera donc la liste des ordres du premier programme. Il est à la tête des actions physiques du robot, et pourra utiliser `ROBOT_avancer`, `ROBOT_intersection` et `ROBOT_urgence`. Toutes ces actions dépendent de codes agissant directement sur les ports de la raspberry. Des vitesses et des sens de rotations sont ainsi associé à chaque action pour chaque moteur. À tout moment, les capteurs de lignes envoient de l'information au programme. Avec cette vision, les choix de virage ou de redressement sont pris.

1.2 Conception préliminaire

1.2.1 Conception Préliminaire Labyrinthe

Conception Préliminaire **Type** Case = **Naturel**
Type Contenu = **Tableau[1..4] de Case**
Type Labyrinthe = **Structure**
lesConnections : **Tableau[1..MAX] de Case**
entrée : **Case**
sortie : **Case**
fin Structure

Opérations de la Structure Labyrinthe **fonction** labrinthe () : **Labyrinthe**
fonction obtenirCasesAccessibles (lab : **Labyrinthe**) : **Tableau[1..4] de Case**
fonction obtenirEntrée (lab : **Labyrinthe**) : **Case**
fonction obtenirSortie (lab : **Labyrinthe**) : **Case**

Opérations de la structure Case

1.2.2 Conception Préliminaire Case

fonction creerCase() : **Case**
fonction obtenirCaseHaut (uneCase : Case) : **Case**
fonction obtenirCaseBas (uneCase : Case) : **Case**
fonction obtenirCaseGauche (uneCase : Case) : **Case**
fonction obtenirCaseDroite (uneCase : Case) : **Case**
fonction estAccessible(case1, case2 : **Case**) : **Booléen**

1.2.3 Conception Préliminaire ordresRobot

Définitions des types
Type ORD_Position = Case
Enumération ORD_Ordre = {AV, TD, TG, . }
Enumération ORD_Orientation = {Nord, Sud, Est, Ouest, Invalide}

Signatures des fonctions

Fonctions de motricité
fonction ORD_avancer (**Position** position, **Orientation** orientation, **Position**** casesAccessibles) : **Position**
fonction ORD_tournerGauche (**Position** position, **Orientation*** orientation, **Position**** casesAccessibles) : **Position**
fonction ORD_tournerDroite (**Position** position, **Orientation*** orientation, **Position**** casesAccessibles) : **Position**

Fonctions de vérification **fonction** estAccessible (**Position*** positionActuelle, **Position*** positionVoisine) : **Booléen**
fonction estIntersection (**Position*** position, **Labyrinthe*** labyrinthe) : **Booléen**
fonction estVirage (**Position*** position, **Orientation** orientation, **Labyrinthe*** labyrinthe) : **Booléen**

Fonctions d'initialisation et principale
fonction ORD_initialisation (**Orientation*** orientation) : **Position**
fonction ORD_obtenirOrdres (**Position*** chemin) : **Ordre***

Fonctions de décision **fonction** ORD_calculerOrientation (**Position** caseActuelle, **Position** caseSuivante) : **Orientation**
fonction ORD_tournerVers (**Orientation** orientationActuelle, **Orientation** orientationCible) : **Ordre**

1.3 Conception détaillée

1.3.1 SDD Labyrinthe

Structures de Données
Type LAB_Labyrinthe = **Structure**

lesConnections : **Tableau[**1..MAX**] de** CASE_Case
entree : **NaturelNonNul**
sortie : **NaturelNonNul**
largeur : **NaturelNonNul**
finstructure

Conception Préliminaire Labyrinthe
fonction LAB_creerLabyrinthe (l : **NaturelNonNul**, entree, sortie : **Chaine de caracteres) :** LAB_Labyrinthe
fonction LAB_obtenirEntree (lab : LAB_Labyrinthe) **:** CASE_Case
fonction LAB_obtenirSortie (lab : LAB_Labyrinthe) **:** CASE_Case
fonction LAB_obtenirLargeur (lab : LAB_Labyrinthe) **: NaturelNonNul**
fonction LAB_iemeCase (lab : LAB_Labyrinthe, iCase : **NaturelNonNul) :** CASE_Case
fonction LAB_obtenirCasesAccessibles (lab : LAB_Labyrinthe, uneCase : CASE_Case) **: Tableau[**1..4**] de** CASE_Case
fonction LAB_obtenirDirectionEntree (lab : LAB_Labyrinthe) **: Chaine de caracteres**
fonction LAB_obtenirDirectionSortie (lab : LAB_Labyrinthe) **: Chaine de caracteres**

Conception Détaillée Labyrinthe
fonction LAB_creerLabyrinthe (l : **NaturelNonNul**, entree, sortie : **Chaine de caracteres) :** LAB_Labyrinthe
Déclaration lab : LAB_Labyrinthe
debut
lab ← **Tableau[**1..l*l**] de** CASE_Case
lab.entree ← entree
lab.sortie ← sortie
lab.largeur ← largeur
retourner lab
fin
fonction LAB_obtenirEntree (lab : LAB_Labyrinthe) **:** CASE_Case
debut
retourner lab.entree
fin
fonction LAB_obtenirSortie (lab : LAB_Labyrinthe) **:** CASE_Case
debut
retourner lab.sortie
fin

fonction LAB_obtenirLargeur (lab : LAB_Labyrinthe) **: NaturelNonNul**
debut
  retourner lab.largeur
fin
fonction LAB_iemeCase (lab : LAB_Labyrinthe, iCase : **NaturelNonNul) :** CASE_Case
  Déclaration res : Case
debut
  res ← lab.lesConnections[iCase]
  retourner res
fin
fonction LAB_obtenirCasesAccessibles (lab : LAB_Labyrinthe, iCase : **NaturelNonNul) :** Case
  Déclaration res : Case
debut
  res ← LAB_iemeCase(lab, iCase)
  retourner res
fin
fonction LAB_obtenirDirectionEntree (lab : LAB_Labyrinthe) **:** CASE_Case
  Déclaration res : Case
debut
  res ← lab.entree[1]
  retourner res
fin
fonction LAB_obtenirDirectionSortie (lab : LAB_Labyrinthe) **:** CASE_Case
  Déclaration res : Case
debut
  res ← lab.sortie[1]
  retourner res
fin

1.3.2 SDD Case

Structures de Données
Type Case = **Structure**
  Numero : **NaturelNonNul**
  Haut : **NaturelNonNul**
  Gauche : **NaturelNonNul**

Bas : **NaturelNonNul**
Droite : **NaturelNonNul**
finstructure

Conception Préliminaire Case
fonction CASE_creerCase (n : **NaturelNonNul**) **:** Case_Case
fonction CASE_obtenirNumeroCase (c : CASE_Case) **: NaturelNonNul**
fonction CASE_obtenirHaut (c : CASE_Case) **: NaturelNonNul**
fonction CASE_obtenirGauche (c : CASE_Case) **: NaturelNonNul**
fonction CASE_obtenirBas (c : CASE_Case) **: NaturelNonNul**
fonction CASE_obtenirDroite (c : CASE_Case) **:** Case
procédure CASE_fixerHaut (**E/S** c : CASE_Case **E** num : **NaturelNonNul**)

procédure CASE_fixerGauche (**E/S** c : CASE_Case **E** num : **NaturelNonNul**)
procédure CASE_fixerBas (**E/S** c : CASE_Case **E** num : **NaturelNonNul**)
procédure CASE_fixerDroite (**E/S** c : CASE_Case **E** num : **NaturelNonNul**)
fonction CASE_obtenirCasesAccessibles (c : CASE_Case) **: Tableau[**1..4**] de NaturelNonNul**

Conception Détaillée Case
fonction CASE_creerCase (n : **NaturelNonNul**) **:** Case_Case
Déclaration c : CASE_Case
debut
c.Numero ← n
retourner c
fin
fonction CASE_obtenirNumeroCase (CASE : CASE_Case) **: NaturelNonNul**

debut
retourner c.Numero
fin
fonction CASE_obtenirHaut (CASE : CASE_Case) **: NaturelNonNul**
debut
retourner c.Haut
fin
fonction CASE_obtenirGauche (CASE : CASE_Case) **: NaturelNonNul**
debut

retourner c.Gauche
fin
fonction CASE_obtenirBas (CASE : CASE_Case) **: NaturelNonNul**
debut
retourner c.Bas
fin
fonction CASE_obtenirDroite (CASE : CASE_Case) **: NaturelNonNul**
debut
retourner CASE.Droite
fin
procédure CASE_fixerHaut (**E/S** c : CASE_Case **E** num : **NaturelNonNul)**
debut
c.Haut ← num
fin
procédure CASE_fixerGauche (**E/S** c : CASE_Case **E** num : **NaturelNon-Nul)**
debut
c.Gauche ← num
fin
procédure CASE_fixerBas (**E/S** c : CASE_Case **E** num : **NaturelNonNul)**
debut
c.Bas ← num
fin
procédure CASE_fixerDroite (**E/S** c : CASE_Case **E** num : **NaturelNon-Nul)**
debut
c.Droite ← num
fin
fonction CASE_obtenirCasesAccessibles (c : CASE_Case) **: Tableau[**1..4**] de NaturelNonNul**

Déclaration res : **Tableau[**1..4**] de NaturelNonNul**

debut
res[1] ← CASE_obtenirHaut(c)
res[2] ← CASE_obtenirGauche(c)
res[3] ← CASE_obtenirBas(c)
res[4] ← CASE_obtenirDroite(c)
retourner res
fin

1.3.3 Conception Détaillée ordres

fonction CASE_obtenirParNumero (**Entier** numero, **LAB_Labyrinthe** labyrinthe) : **ORD_Position***
début

si numero $<$ 0 **ou** numero $\geq$ largeur $\times$ largeur **alors**
retourner NULL
fin si
retourner labyrinthe.lesConnections[numero]

fin

fonction estAccessible (**ORD_Position*** positionActuelle, **ORD_Position*** positionVoisine) : **Booléen**
début

retourner positionVoisine $\neq$ NULL

fin

fonction ORD_avancer (**ORD_Position*** position, **ORD_Orientation** orientation, **LAB_Labyrinthe** labyrinthe, **Entier** largeurLabyrinthe) : **ORD_Position***
début

tant que vrai **faire**
voisinNumero $\leftarrow$ -1
si orientation = Nord **alors**
voisinNumero $\leftarrow$ CASE_obtenirHaut(position)
sinon si orientation = Sud **alors**
voisinNumero $\leftarrow$ CASE_obtenirBas(position)
sinon si orientation = Est **alors**
voisinNumero $\leftarrow$ CASE_obtenirDroite(position)
sinon si orientation = Ouest **alors**
voisinNumero $\leftarrow$ CASE_obtenirGauche(position)
fin si
voisinTrouve $\leftarrow$ CASE_obtenirParNumero(voisinNumero, labyrinthe)
si voisinTrouve = NULL **ou** ORD_estIntersection(voisinTrouve, labyrinthe) **ou**
ORD_estVirage(voisinTrouve, orientation, labyrinthe, largeurLabyrinthe) **alors**
retourner voisinTrouve
fin si

position ← voisinTrouve
fin tant que

fin

fonction ORD_tournerGauche (**ORD_Position*** position, **ORD_Orientation** orientation, **LAB_Labyrinthe** labyrinthe, **NaturelNonNul** largeurLabyrinthe) : **ORD_Position***
début

si orientation = Nord **alors**
orientation ← Ouest
sinon si orientation = Ouest **alors**
orientation ← Sud
sinon si orientation = Sud **alors**
orientation ← Est
sinon si orientation = Est **alors**
orientation ← Nord
fin si
retourner ORD_avancer(position, orientation, labyrinthe, largeurLabyrinthe)

fin

fonction ORD_tournerDroite (**ORD_Position*** position, **ORD_Orientation** orientation, **LAB_Labyrinthe** labyrinthe, **NaturelNonNul** largeurLabyrinthe) : **ORD_Position***
début

si orientation = Nord **alors**
orientation ← Est
sinon si orientation = Est **alors**
orientation ← Sud
sinon si orientation = Sud **alors**
orientation ← Ouest
sinon si orientation = Ouest **alors**
orientation ← Nord
fin si
retourner ORD_avancer(position, orientation, labyrinthe, largeurLabyrinthe)

fin

fonction ORD_initialisation (**LAB_Labyrinthe** labyrinthe, **ORD_Orientation** orientation) : **ORD_Position***
début

entree ← CASE_obtenirParNumero(labyrinthe.entree, labyrinthe)
si entree = NULL **alors**
afficher "Erreur : entrée invalide dans le labyrinthe."
quitter le programme
fin si
si estAccessible(entree, CASE_obtenirParNumero(CASE_obtenirHaut(entree), labyrinthe)) **alors**
orientation ← Nord
sinon si estAccessible(entree, CASE_obtenirParNumero(CASE_obtenirBas(entree), labyrinthe)) **alors**
orientation ← Sud
sinon si estAccessible(entree, CASE_obtenirParNumero(CASE_obtenirGauche(entree), labyrinthe)) **alors**
orientation ← Ouest
sinon si estAccessible(entree, CASE_obtenirParNumero(CASE_obtenirDroite(entree), labyrinthe)) **alors**
orientation ← Est
sinon
afficher "Erreur : aucune direction accessible."
quitter le programme
fin si
retourner entree

fin

fonction ORD_calculerOrientation (**ORD_Position*** caseActuelle, **ORD_Position*** caseSuivante, **NaturelNonNul** largeurLabyrinthe) : **ORD_Orientation**
début

numActuelle ← CASE_obtenirNumeroCase(caseActuelle)
numSuivante ← CASE_obtenirNumeroCase(caseSuivante)
si numSuivante = numActuelle - largeurLabyrinthe **alors**
retourner Nord
sinon si numSuivante = numActuelle + largeurLabyrinthe **alors**
retourner Sud
sinon si numSuivante = numActuelle + 1 **alors**
retourner Est
sinon si numSuivante = numActuelle - 1 **alors**

```
        retourner Ouest
    fin si
    retourner Invalide

fin

fonction ORD_tournerVers (ORD_Orientation orientationActuelle, ORD_Orientation
orientationCible) : ORD_Ordre
début

    si orientationActuelle = orientationCible alors
        retourner AV
    fin si
    si (orientationActuelle = Nord et orientationCible = Ouest) ou
        (orientationActuelle = Ouest et orientationCible = Sud) ou
        (orientationActuelle = Sud et orientationCible = Est) ou
        (orientationActuelle = Est et orientationCible = Nord) alors
        retourner TG
    fin si
    retourner TD

fin

fonction ORD_estIntersection (ORD_Position* position, LAB_Labyrinthe
labyrinthe) : Booléen
début

    voisinsAccessibles ← 0
    si estAccessible(position,
CASE_obtenirParNumero(CASE_obtenirHaut(position), labyrinthe)) alors
        voisinsAccessibles ← voisinsAccessibles + 1
    fin si
    si estAccessible(position,
CASE_obtenirParNumero(CASE_obtenirBas(position), labyrinthe)) alors
        voisinsAccessibles ← voisinsAccessibles + 1
    fin si
    si estAccessible(position,
CASE_obtenirParNumero(CASE_obtenirGauche(position), labyrinthe)) alors
        voisinsAccessibles ← voisinsAccessibles + 1
    fin si
    si estAccessible(position,
CASE_obtenirParNumero(CASE_obtenirDroite(position), labyrinthe)) alors
```

voisinsAccessibles ← voisinsAccessibles + 1
fin si
retourner voisinsAccessibles > 2
fin

fonction ORD_estVirage (**ORD_Position*** position, **ORD_Orientation** orientation, **LAB_Labyrinthe** labyrinthe, **NaturelNonNul** largeurLabyrinthe) : **Booléen**
début
voisinHaut ← CASE_obtenirParNumero(CASE_obtenirHaut(position), labyrinthe)
voisinBas ← CASE_obtenirParNumero(CASE_obtenirBas(position), labyrinthe)
voisinGauche ← CASE_obtenirParNumero(CASE_obtenirGauche(position), labyrinthe)
voisinDroite ← CASE_obtenirParNumero(CASE_obtenirDroite(position), labyrinthe)
directionsAccessibles ← 0
si voisinHaut **alors** directionsAccessibles ← directionsAccessibles + 1 **fin si**
si voisinBas **alors** directionsAccessibles ← directionsAccessibles + 1 **fin si**
si voisinGauche **alors** directionsAccessibles ← directionsAccessibles + 1 **fin si**
si voisinDroite **alors** directionsAccessibles ← directionsAccessibles + 1 **fin si**
si directionsAccessibles = 2 **alors**
si (orientation = Nord **ou** orientation = Sud) **et** (voisinGauche **ou** voisinDroite) **alors**
retourner vrai
fin si
si (orientation = Est **ou** orientation = Ouest) **et** (voisinHaut **ou** voisinBas) **alors**
retourner vrai
fin si
fin si
retourner faux
fin

fonction genererEtAfficherOrdres (**Tableau[ORD_Position*]** chemin, **NaturelNonNul** tailleChemin, **LAB_Labyrinthe** labyrinthe) : **Vide**
début

```
ordres ← tableau de taille (2 × tailleChemin + 2)
ordreIndex ← 0
orientationActuelle ← ORD_calculerOrientation(chemin[0], chemin[1],
labyrinthe.largeur)
si ORD_estIntersection(chemin[0], labyrinthe) ou
ORD_estVirage(chemin[0], orientationActuelle, labyrinthe, labyrinthe.largeur)
alors
    ordres[ordreIndex] ← AV
    ordreIndex ← ordreIndex + 1
fin si
pour i ← 1 à tailleChemin - 1 faire
    orientationSuivante ← ORD_calculerOrientation(chemin[i - 1],
chemin[i], labyrinthe.largeur)
    si orientationActuelle ≠ orientationSuivante alors
        ordres[ordreIndex] ← ORD_tournerVers(orientationActuelle,
orientationSuivante)
        ordreIndex ← ordreIndex + 1
        ordres[ordreIndex] ← AV
        ordreIndex ← ordreIndex + 1
        orientationActuelle ← orientationSuivante
    sinon
        si ORD_estIntersection(chemin[i], labyrinthe) ou
ORD_estVirage(chemin[i], orientationActuelle, labyrinthe, labyrinthe.largeur)
alors
            ordres[ordreIndex] ← AV
            ordreIndex ← ordreIndex + 1
        fin si
    fin si
fin pour
ordres[ordreIndex] ← FIN
pour i ← 0 tant que ordres[i] ≠ FIN faire
    selon ordres[i] faire
        cas AV : afficher "AV"
        cas TG : afficher "TG"
        cas TD : afficher "TD"
        par défaut : rien
    fin selon
fin pour
libérer ordres
libérer labyrinthe.lesConnections
```

libérer labyrinthe

fin

1.4 Tests

Nous n'avons pas réalisés de tests propres tels qu'on les a vus en cours avec Cunit. Mais pour chaque TAD nous avons réalisé un module de tests. Par exemple pour le TAD pile, nous avons implémenter un programme qui empile, dépile tout en affichant l'état de la pile. Ceci nous permet de nous assurer que nos TAD fonctionnent avant de les utiliser dans la recherche de chemin.

1.5 Retour sur la partie algorithmique

Dans la partie algorithmique, nous avons rencontré plusieurs difficultés tout au long du projet.

Pour commencer, certains tests ont été réalisés, néanmoins il ne s'agissait pas de tests unitaires au sens strict. Les tests que nous avons faits avaient avant tout pour but de visualiser nos programmes au fur et à mesure du développement, comme par exemple la création d'un labyrinthe fictif pour vérifier la bonne génération des ordres pour un chemin donné. Bien que ces tests nous aient été utiles, la détection d'erreurs plus spécifiques est à revoir.

En ce qui concerne l'algorithme de résolution du labyrinthe, sa logique est correcte et l'algorithme est correct. Malheureusement, nous avons rencontré de nombreux problèmes au moment de l'implémentation, notamment concernant les allocations dynamiques de mémoire. Ce problème est avant tout lié au manque de maîtrise de certains concepts du langage C. Manquant de temps, nous n'avons malheureusement pas pu compléter cette partie.

Enfin, le problème principal que nous avons pu relever dans ce projet est le manque d'organisation et la gestion du temps. Nous avions identifié ces problématiques dès le départ, mais leur impact s'est surtout fait ressentir pendant les dernières semaines avant le rendu, quand les problèmes ont commencé à s'accumuler. Cette difficulté nous a montré l'importance d'une meilleure organisation pour les projets à venir.

Pour conclure, ces problèmes nous ont permis de nous rendre compte de plusieurs points. Ils ont mis en avant l'importance des tests unitaires, d'une meilleure planification, et des points à revoir en programmation.

2 Électronique

2.1 Description des composants

Nous avons séparé le câblage en trois pôles :
— Un pôle Son
— Un pôle Vision
— Un pôle Moteur
Des *wagos* sont utilisés pour centraliser le GROUND et les alimentations de la RaspBerry lorsque c'est nécessaire.

Pôle Son
Le premier pôle comprend le système émetteur-récepteur d'ultrasons et le buzzer. Placé à l'avant, le rôle de ce premier est de calculer la distance qui sépare l'avant du Robot d'un potentiel obstacle. Si cette distance descend en dessous d'un seuil défini, le Robot doit s'arrêter. Le buzzer est quand à lui utilisé pour émettre un signal sonore, notament si le Robot s'arrête lorsqu'il rencontre un obstacle.

Pôle Vision
Le deuxième pôle se compose de l'écran LCD, qui est placé sur le dessus du Robot et des trois capteurs de ligne. Le rôle de l'écran LCD est d'afficher des informations. Dans le cadre de notre projet, nous l'utilisons pour indiquer les différents ordres que le Robot doit suivre, ainsi que son état d'évolution dans le labyrinthe (lorsqu'il est sorti par exemple). Nous l'avons également utilisé pour localiser des erreurs lors de nos tests.

Ensuite, les capteurs de ligne sont faits pour envoyer un signal en fonction de la luminosité qu'ils détectent. C'est le contraste entre les lignes et le reste des tuiles qui les rendent intéressants dans la détection de ligne (comme leur nom l'indique). Grâce à ces capteurs, nous pouvons détecter les situations dans lesquelles se trouvent le Robot afin qu'il réagisse en conséquence.

Pôle Moteur

Le dernier pôle est bien évidemment dédié aux moteurs. Ces derniers sont placés à l'arrière du Robot, car c'était à cet emplacement que cela nous paraissait le plus stable, avec une roue ronde à l'avant. La carte L293D est vissée aux parois du châssis afin de faciliter son accès pour les interventions que nous avons dû faire dessus.

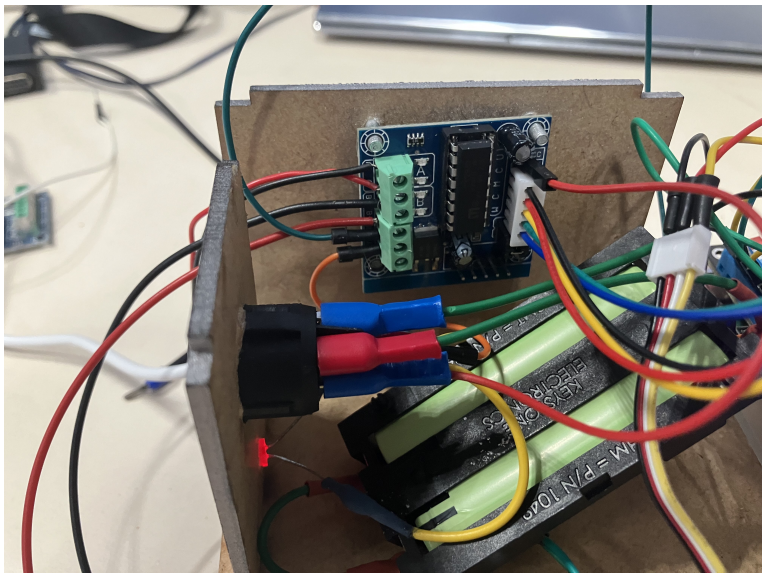

Table 2 – Emplacement des moteurs

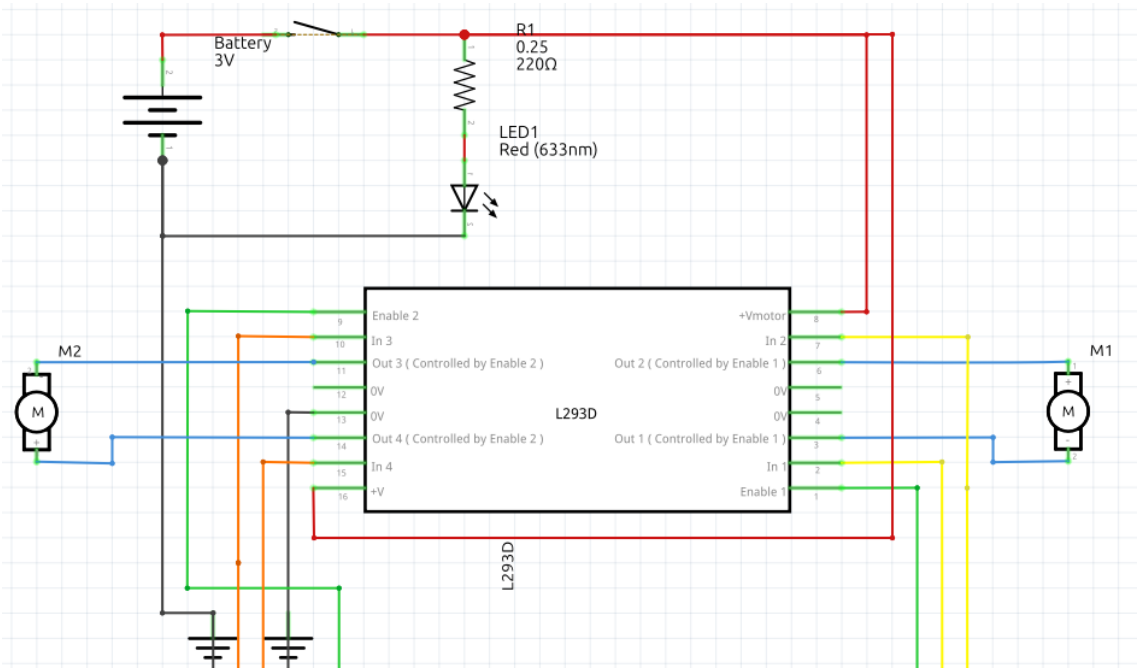

TABLE 3 – Schéma électrique du pôle Moteur

2.2 Schéma électrique

Le schéma électrique a été fait sur *Fritzing* au début du projet.

Au niveau du choix des branchements, nous avons le plus possible essayé de rapprocher les numéros des GPIO lorsqu'ils étaient utilisés par le même composant, ou un composant similaire. Par exemple, les pins IN1 et IN2 du moteur 1 sont branchées sur les GPIO 20 et 21 tandis que les capteurs positionnés à droite et à gauche sont branchés sur les GPIO 7 et 8.
Cependant, nous avons tout de même privilégié un branchement tel que le schéma électrique numérique soit le plus lisible possible tant que cela était possible. Cela permet une meilleure compréhension générale du câblage, et rend la résolution de problèmes plus efficace.

Lorsque nous avons monté le Robot, nous avons d'abord fait tous les branchements nécessaires avant de percer le châssis et de le fermer. Ainsi, l'entièreté de nos câbles ont pu rentrer dans le Robot (à part

ceux reliés aux moteurs) et les interventions sur les branchements était facilitées.

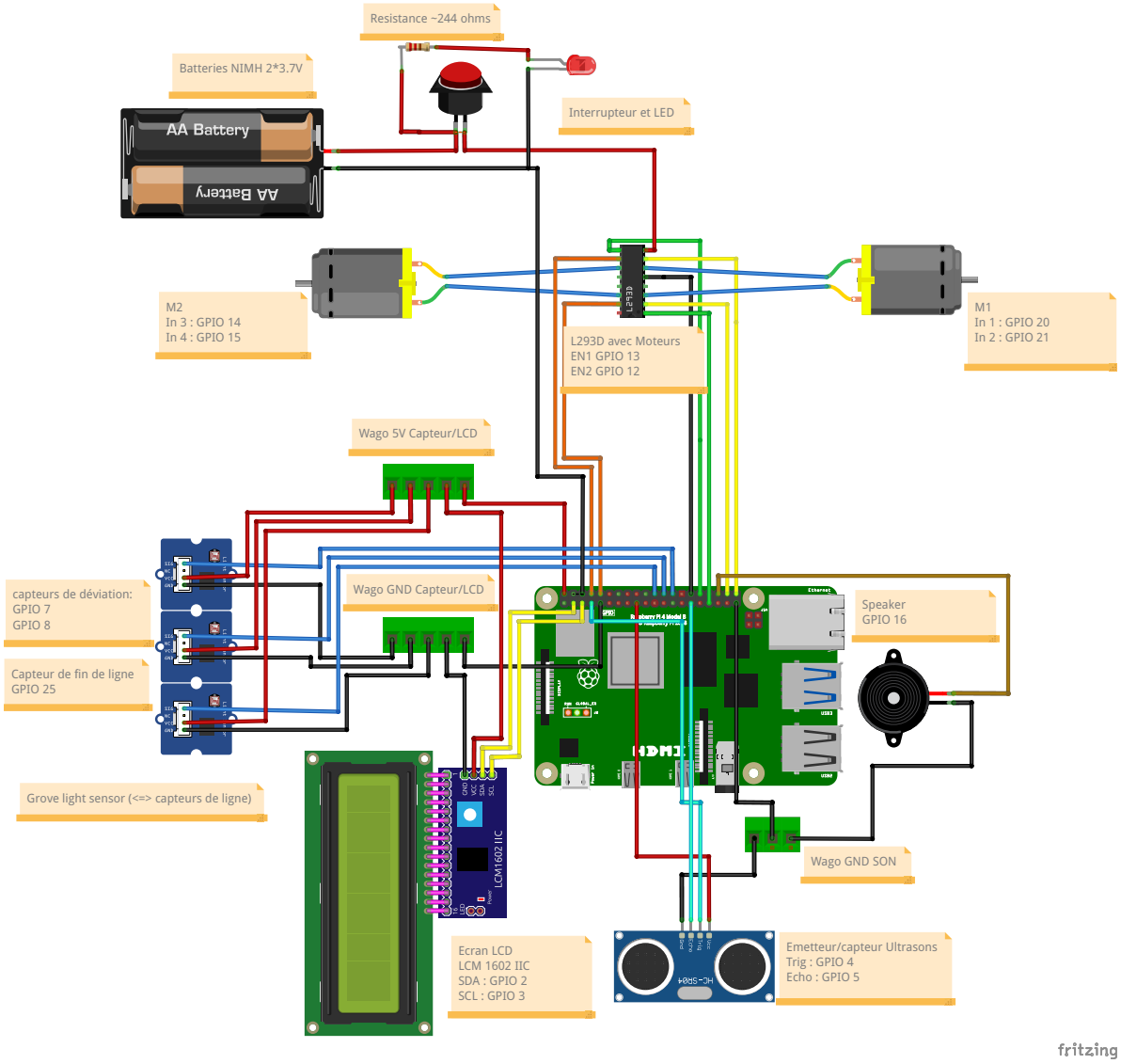

Table 4 – Schéma électrique

2.3 Positionnement des capteurs

Nous avons pris la décision de placer trois capteurs détecteurs de ligne sur notre robot. Cette décision, nous avons pris du temps à la prendre et en comparant avec d'autres groupes, on remarque que beaucoup ont pris des décisions différentes.

Pourquoi trois capteurs, et quels sont leur rôle ? Un capteur au centre (dans le trou à l'avant centre du châssis) et les deux autres à l'avant et dirigé vers les côtés. Le capteur au centre a un rôle de contrôle : "Suis-je sur la ligne ?". Si le robot est bien sur la ligne, aucun soucis. Sinon, les deux autres capteurs vont nous aider à savoir dans quelle situation le robot se trouve :

— Cas "classique" : CPTR_estSurLaLigne
  le robot est sur la ligne, sur une ligne droite. Dans ce cas le capteur central est à l'état 1 et les deux capteurs extérieurs à l'état 0. Ce qui donne :
  0 1 0
— Intersection : CPTR_estSurUneIntersection
  le robot est sur la ligne et arrive sur un virage à droite :
  0 1 1
  OU le robot est sur la ligne et arrive sur un virage à gauche :
  1 1 0
  OU le robot est sur la ligne et arrive sur une intersection (en T ou en croix)
  1 1 1
— Trop à gauche : CPTR_estTropAGauche
  le robot dévie vers la gauche en ligne droite :
  0 0 1
— Trop à droite : CPTR_estTropADroite
  le robot dévie vers la droite en ligne droite :
  1 0 0

Nous avons utilisé cette distinction de cas pour la conception des fonctions liées à l'avancement du robot dans le labyrinthe.

2.4 Algorithme de déplacement

```
procedure ROBOT_evolutionRobot ()
    Déclaration :  prochaineAction : Chaine de caractères
                   etat : ROBOT_EtatDAvancement
début
   etat ← Avancer
   tant que prochaineAction ≠ FIN faire
      lire (prochaineAction)
      suivant etat faire
         cas où Avancer faire
            ROBOT_avancer (etat)
         fin
         cas où Intersection faire
            ROBOT_intersection (etat, prochaineAction)
         fin
         cas où Urgence faire
            ROBOT_urgence ()
         fin
      fin
   fin
fin
```

Algorithme de ROBOT_evolutionRobot

Pour élaborer l'algorithme de déplacement du robot nous avons considéré qu'il pouvait se retrouver dans 4 états différents :

— En avancée
— Sur une intersection
— En situation d'urgence
— Sorti du labyrinthe

Nous avons donc une variable *etat* du type **ROBOT_EtatDAvancement** qui est mise à jour à chaque tour d'une boucle qui ne s'arrête que lorsque la suite d'ordre que nous lui fournissons s'achève. Selon la valeur de cette variable *etat*, différentes fonctions sont appelées afin de

faire évoluer le Robot.

La première est la fonction ROBOT_avancer, qui met les moteurs en marche avant et qui adapte la trajectoire du robot grâce aux capteurs.

Ensuite, la fonction ROBOT_intersection agit selon l'ordre lu par l'algorithme principal. Elle fait tourner le Robot à droite ou à gauche ou bien le fait continuer tout droit. Le Robot effectue un virage sur place, nous avons donc paramétré un temps d'attente afin de positionner correctement le centre de rotation du robot sur l'intersection.

La fonction, ROBOT_urgence, est la fonction qui est appelée lorsqu'un obstacle est détecté. Son rôle est d'arrêter les moteurs et d'allumer le buzzer.

2.5 Problèmes recontrés

La première difficulté que nous pouvons citer porte sur l'émetteur-récepteur d'ultrasons. Malheureusement, l'algorithme utilisé ne parvenait pas à sortir d'une boucle infinie ce qui rendait le Robot incapable de suivre les autres programmes. Nous avons tenté de paralléliser ce dispositif mais sans succès, c'est pourquoi nous n'avons pas pu présenter cette option lors de la soutenance.

Pendant une grande partie du projet, nous avions le sentiment d'avoir une longueur de retard sur les autres groupes à propos de la partie électronique. Ce retard était dû aux nombreuses discussions que nous avions eu en amont sur la conception du Robot. Nous avons discuté du nombre de capteurs, de la position des moteurs et d'autres dispositifs pendant plusieurs séances avant de commencer à percer et à souder. Au final, cela ne nous a pas desservi ; effectivement, après le montage, tous le câblage rentrait parfaitement dans le châssis et la plupart des composants ont fonctionné du premier coup. De plus, la plupart des erreurs rencontrées venaient des codes utilisés.

Cependant, lors de nos premiers esssais, les moteurs ne répondaient

pas. C'était une simple erreur de câblage sans gravité (aucun matériel n'a été dégradé), mais avec la fatigue, nous avons mal vérifié l'ensemble des connectiques et le problème ne fût résolu que lors de la séance d'essais suivante.

Avec ce retard accumulé, nous avons manqué de temps pour les derniers réglages, c'est pourquoi un dernier bug subsistait lors de la présentation du Robot.

3 Voies d'amélioration

Notre Robot possède de nombreux inconvénients. Tout d'abord, le programme de recherche du plus court chemin n'est pas terminé, ce qui constitue un gros désavantage. Ensuite, notre choix de représentation du Labyrinthe en C convient pour cette application simple et cadrée, mais elle n'est pas très portable. Une représentation plus complexe comme les graphes aurait surement été préférable pour faciliter son utilisation.

Ensuite, le système de redressement n'est pas au point car il gêne la détection d'intersection. De plus, la détection de sa condition d'arrêt laisse à désirer. Nous aurions pu utiliser des capteurs supplémentaires afin de séparer la détection d'intersection du redressement ou alors rechercher un algorithme plus complexe servant à remettre le Robot dans une position correcte à chaque grosse déviation. De plus, comme mentionné précédemment, le système de détection d'obstacle ne fonctionne pas du tout.

Une autre voie d'amélioration possible porte sur notre méthode de travail. Afin de rendre la résolution de problème plus efficace, il aurait fallu définir en amont un protocole à suivre qui aiderait à revoir l'ensemble des éléments à l'origine d'un potentiel problème rencontré. Avec une telle méthode, nous aurions peut-être perdu moins de temps à chercher où étaient nos erreurs, surtout que la plupart du temps, elles sont très facilement corrigées.

Cette remarque vaut notamment pour la partie électronique et le montage du Robot, pour la partie algorithmie, il faudrait coder un ensemble de tests avant de dévelloper nos programmes (comme le suggère le cycle en V).